

ManPy: an open-source software tool for building discrete event simulation models of manufacturing systems

Georgios Dagkakis^{*,†}, Ioannis Papagiannopoulos and Cathal Heavey

Enterprise Research Centre, University of Limerick, Limerick, Ireland

SUMMARY

In this paper, we present a new open-source (OS) software library for building discrete event simulation objects with focus on manufacturing environments. ManPy stands for ‘Manufacturing in Python’ but employs a generic approach that can be extended to other types of business processes such as services, logistics and supply chain management. It is written in Python and makes use of the SimPy library to implement a process interaction world view. The goal in developing ManPy is to provide an expandable OS layer of well-defined manufacturing objects, which can be used by users with multiple levels of expertise in discrete event simulation, namely, a super user and an industrial engineer. This object repository follows a structured architecture, allowing developers to extend it, exchange ideas and methodologies, with the goal of forming an OS community. We explain how ManPy is developed on SimPy, present its architecture and give examples of its utilization. We also give insight of how this work is planned to progress in order to attract software developers, modellers and practitioners in an OS community. Copyright © 2015 John Wiley & Sons, Ltd.

Received 10 February 2015; Revised 18 May 2015; Accepted 17 July 2015

KEY WORDS: discrete event simulation; open source; Python; SimPy; ManPy

1. INTRODUCTION

Discrete event simulation (DES) is arguably one of the most widely accepted and used operations research (OR) methodologies [1–3]. Although DES is traditionally computationally expensive [4, 5], it is often preferred to mathematical methods due to its flexibility [6–8]. Indeed, in a review of simulation papers published in the literature of 281 peer-reviewed papers published between 1997 and 2006 [9], researchers found that DES was the most popular OR technique used.

Commercial off-the-shelf (COTS) DES software has been around for several decades and has contributed considerably to the diffusion of DES in the academic and industrial communities. The strength of such packages lies in the fact that they provide the user with graphical tools for modelling, debugging and experimentation, with advanced visual facilities that enhance model development [10, 11]. On the other hand, such software introduces multidimensional cost that deters companies and especially small and medium enterprises from adopting DES-based solutions in their decision support [12]. Moreover, the lack of access to the source code often hinders important modelling aspects such as flexibility and reusability [13].

Given the previous issues with COTS tools, an open-source (OS) DES library could provide a competent comparative tool, providing an open and lower-cost tool, if tailored to be used by users with multiple levels of expertise in DES. From our review of the state of the art in OS DES [14], which is briefly presented in Section 2 of this current paper, we found that most tools are targeted towards users with high programming expertise. Our objective was not only to produce a functional

^{*}Correspondence to: Georgios Dagkakis, Enterprise Research Centre, University of Limerick, Limerick, Ireland.

[†]E-mail: georgios.dagkakis@ul.ie

OS simulation library, but to make the library accessible to different levels of users. This is the rationale for using the Python language, a language that is being widely used by engineers [15]. In this paper, we describe the ManPy library built using Python and SimPy. The main design challenge in developing ManPy is to encode ManPy objects from the processes used in SimPy models. These objects can then be used by DES modellers in developing simulation models, in a similar manner to users of COTS DES tools. In our design, we create an application programming interface (API) for the creation of extendable OS DES components. ManPy as a simulation engine is modular so it can be integrated to a wide set of decision support platforms. In this work, we aim to research if it is possible for expert users and engineers to work with this DES library. To our knowledge, this is the first OS DES library designed to research this hypothesis, and this is a focal scientific contribution of the ManPy library.

ManPy is part of a broader OS simulation-based decision support platform being developed within the FP7 project titled 'Decision support in Real-time for Efficient Agile Manufacturing' (DREAM, <http://dream-simulation.eu/> accessed 2 February 2015) which started in October of 2012. ManPy is the simulation engine being built within the DREAM platform, and it constitutes a standalone project. The DREAM code, including ManPy, is accessible at the project repository in GitHub (<https://github.com/nexedi/dream> accessed 2 February 2015).

The remainder of this paper is structured as follows: in Section 2, we provide a more in-depth discussion on the motivation for the development of the OS library. In Section 3, we give a brief presentation of SimPy, the OS library on which the ManPy OS tool has been developed and also present the overall design for ManPy. This will allow the reader to understand what SimPy offers and comprehend why ManPy is a separate project. Section 4 contains a presentation of ManPy. We start by explaining the design and architecture of the tool and proceed giving two examples of models that use ManPy objects. In Section 5, we describe the approach used for code verification and consistency establishment between different versions. Next, in Section 6, we present an overview of three case studies where the tool is being utilized. Finally, Section 7 is a discussion about how an OS DES project targeted at manufacturing can reach users of different levels and form a community that develops the object repository. We conclude with a summary of the presented material and notes on the future steps of our developmental process.

2. MOTIVATION

The first step in our study was to conduct a review of the state of the art in OS DES. Our review [14] covered scientific literature and OS projects that deal with DES. Our search revealed 44 OS DES projects. In order to be able to evaluate them and extract information on the state of the art in OS DES, we defined nine evaluation criteria. These were as follows:

- *Language*: the general purpose language that the tool is written in.
- *License*: the OS license that the tool is distributed under.
- *Latest release*: the time that the latest version of the software was released.
- *Latest update*: the time that the project was last updated in any way.
- *Simulation technique*: we reviewed DES tools, but some of them combined DES with another technique, for example, agent-based simulation, so we noted this.
- *Domain*: the domain of problems that the tool is focused on, communications, computing, manufacturing and so on.
- *Documentation*: means that the project provides for documentation. These can include documents, online tutorials, videos and so on.
- *Communication*: means that the project uses to facilitate the communication between users.
- *Version control*: the means, if any, that the project uses in order to manage different versions of the code and support documents.

This work led to numerous interesting results. First of all, the large number of projects shows that there is ongoing interest in the domain. However, most of the projects seem to be abandoned or at least fail to attract a community of users; this is evidenced by examining archives of communication portals, such as mailing lists or fora. Another deficiency we identified is that many projects seem to

be targeted solely to computer programmers. This is opposite to COTS simulation packages within manufacturing, whose primary targeted user is the industrial engineer, that is, someone who is more capable of modelling rather than programming. Sometimes, even the installation of the OS tool is a strenuous task that requires manual deployment of libraries that it depends on. So, the user has to possess advanced programming expertise even to model very simple systems that would require only connecting and parameterizing standard objects of a COTS simulation package.

Another interesting finding was that in the domain of modelling of computer and telecommunication networks, OS DES seems to be much more successful than other OR domains such as manufacturing or logistics. In this domain, projects like OMNeT++ [16] and NS-3 [17] maintain communities of numerous users. Moreover, there is existence of other projects or plug-ins that are built on top of the aforementioned projects focused on specific application areas. For example, OverSim [18] is built on top of OMNeT++ to model peer-to-peer networks. Academic literature has also shown interest producing papers that review OS projects in this domain [19, 20]. We believe that the growth of this domain is partly due to the fact that this topic lies by default inside the interest of a software developer. This way, it does stimulate the 'itch in the developer's head' that Raymond [21] identified as one of the main driving forces of OS development. Unfortunately, this does not seem to be the case in manufacturing, where a background in engineering seems to be essential.

The previous analysis provides motivation to research the development of an OS DES tool that would be focused on supporting engineers but also have modelling flexibility to allow the library to be embedded into decision support systems by software experts. More specifically, we identified two potential categories of users for an OS DES engine:

- *Super user (SU)*: he/she can access the code directly, customize objects flexibly or create completely new ones and embed these simulation objects within decision support systems. He/she needs to have good coding and modelling skills.
- *Industrial engineer (IE)*: he/she can use tailored objects in order to connect them and create a model. Also, he/she may apply some code to customize the behaviour of the model within certain boundaries. Limited coding experience is required at this level, but good modelling skills are essential.

Two options were available to develop an OS DES tool for manufacturing: (i) develop a new library or (ii) adopt an existing library. From examining existing libraries, the three most competent projects we identified were OMNeT++ [16, 22], JaamSim [23] and SimPy [24].

OMNeT++ (<http://www.omnetpp.org/> accessed 2 February 2015) is arguably the most successful OS DES project found. It has an active community of users and has been used in numerous cases and publications (<http://www.omnetpp.org/publications> accessed 2 February 2015). As we already stated, it was also used as the basis of other tools [18, 25]. OMNeT++ is primarily targeted to the modelling of computer and telecommunication networks. It is generic and well implemented, but whereas there have been numerous models and extensions regarding specific issues and protocols of computer and telecommunication networks, we could find few applications in the domains of manufacturing or logistics [26, 27]. Bause *et al.* [28] also report that they could not find much research available about analysis of business processes using the OMNeT++ platform. Moreover, they claim that the mapping of their hierarchical process chains models onto OMNeT++ has been feasible, yet far from being trivial, because complex hierarchies had to be transferred from one view into another view. The second, and maybe most important, reason that would make OMNeT++ incompatible with our needs is its non-permissive license. As it is stated in the OMNeT++ website, the free Academic Public License covers the use of the tool only in academic and educational environments, while a commercial license is required for commercial users.

JaamSim (<http://jaamsim.com/> accessed 2 February 2015) is a tool developed by the Canadian consulting company Ausenco (<http://www.ausenco.com/> accessed 2 February 2015). Although they have been working on their simulation engine for more than a decade [13], the project was released as OS only in 2013 [23]. JaamSim is equipped with an impressive 3-D user interface that can compete against COTS DES software. The project gets daily updates and maintains a forum with several topics. Another asset is that JaamSim as software can be downloaded as a standalone executable (of about 6 to 7 MB for different distributions regarding Windows, OS X and Linux).

At this time, the project does not have a wide community, but it is a recently established project. JaamSim however is licensed under the General Public License, which implies that every project that uses it is obliged to use the same license [29], which is a major constraint for new projects using JaamSim.

SimPy (<http://simpy.readthedocs.org/> accessed 2 February 2015) is a DES framework based on standard Python and has been the tool that we decided to base our work upon. It has an active community of users that communicate mostly via a dedicated mailing list. Documentation is also adequate providing an understanding of SimPy concepts and examples of simple simulation models. SimPy can be used as a low-level simulation language, which provides easiness in the means of handling the parallel progress of processes in simulation time. In this fashion, it is generic and easier to manipulate in order to tailor it to our needs. Another asset is that it is distributed under the Massachusetts Institute of Technology license, which is permissive, giving us flexibility in the way that we license our work. Finally, we found that SimPy was one of the few OS DES tools that got used by academics different than the original authors of the tool [2, 30, 31]

One of the most important reasons that we selected SimPy has been the fact that it is written in Python. As we explained, the aspiration of our work is to develop a library of manufacturing objects that users can customize and exchange at different levels. In order for this library to remain actively updated, it would have to attract modellers who would utilize it, primarily through parameterization and addition of some Python code to create their own objects. As we already stated, DES modelling is challenging [32] and especially modelling of manufacturing systems, which requires a background of industrial engineering. People with such background are often not experts in software engineering. Python provides an intuitive syntax that makes it easier for modellers to write code [33]. This is one of the reasons that engineers have started to consider it as a strong free alternative to MATLAB [34] with Python being actively used in engineering, extended with specialized libraries [15]. In this aspect, a simulation engine written in Python can easily be integrated with other popular Python scientific libraries like SciPy [35] or scikit-learn [36] or even simulation-oriented tools like ScipySim [37], which facilitates heterogeneous simulation, and PySimulator [38], which provides an OS environment to perform simulations with different simulation engines and analysis plug-ins.

Python possesses also specific attributes that enhance the writing of a simulation programme for a user with software expertise. Namely, its flexible type casting facilitates the building of an environment that contains instances of classes that communicate like black boxes. Moreover, Python offers numerous decorators and methods for effective list processing, which is a core element of the DES paradigm. It is not implied that other computer languages do not provide efficient methods for these issues, but the easiness that Python offers in this aspect makes it more attractive even to users that are not computer experts.

The trade-off is that Python, as a scripting language, is by default slower than static languages such as C++ or Java [39]. There have been attempts to enhance the performance of Python generally [40, 41] and SimPy specifically [42]. Cloud deployment of simulation in a simulation-as-a-service (SimaaS) approach [43] can be another solution for a cloud-enabled OS simulation project.

3. SIMPY AND MANPY OVERALL DESIGN

This section is a brief presentation of the SimPy project. The scope is to provide the reader with an understanding of what SimPy is and how ManPy is a different project built on the SimPy library.

SimPy stands for ‘Simulation in Python’ and is a package for process-oriented DES written exclusively in Python. SimPy’s design offers a clean way of handling Python’s generators [24]. A normal function or method in Python (and most programming languages) executes its code from the first line every time it is called. If the method needs to return a value, then the *return* keyword is used after which the method exits. The next time that the method is called, its execution will commence again from the beginning of its code. A generator method in Python is declared like any other method. The difference is that it can make use of the *yield* command in order to return a value and release control. When the method is invoked again, it will not execute its code from the beginning, but from the next line of the last *yield* statement that released control.

Python generators have many uses in large calculations but are also ideal for coding processes that run in parallel in a DES environment. In a process-oriented world view, processes like

customers or servers may take control of a programme to change their state at a specific moment of simulation time when an event occurs (e.g. customer X ended processing in server Y). Then they release control so that other processes who reside in the same 'environment' can take it and yield events that they wait for.

SimPy employs generators cleanly in order to give the means for the user to write simulations. In every model, an instance from the Environment class is created. This is where all the simulation objects reside and can interact with each other. Then SimPy offers two major classes [24]:

- *Process*: models an entity that evolves in simulation time. It is not a formal Python thread but allows processes to run in parallel in the simulation environment.
- *Resource*: models an entity that other entities may compete for (e.g. a server).

SimPy has an active community of users who communicate through a dedicated mailing list. The code is hosted in Bitbucket (<https://bitbucket.org> accessed 2 February 2015) and mercurial (<http://mercurial.selenic.com/> accessed 2 February 2015), which are utilized for version control. The project gets frequent commits, almost on a daily basis, while a new intermediate version (e.g. SimPy3.0.1) is released on a monthly frequency. Documentation on the website is adequate, giving an overview of the language and several results.

SimPy has been around as a project for more than a decade. However, the new SimPy3 version was released last year and introduced many changes in the syntax, rendering models written in SimPy2 not compatible with the new release. There are directions on how to port code from one version to the other, but from our experience, it is not always straightforward. We consider the fact that the project did not keep its backwards compatibility for consistency. Nonetheless, the new version does offer more flexibility. For example, in SimPy2, the programmer had to explicitly define processes, while in a SimPy3 environment, everything is regarded as a process and can act as such. Furthermore, SimPy3 allows the definition of more than one generator method in the same object, which does enhance pliability in coding.

From our interaction with SimPy, we established that it is a solid package, which allows clean coding of simulations. On the other hand, the user has to be a coder even for very simple problems. Even from the very first example that SimPy community offers (<http://simpy.readthedocs.org/en/latest/examples/> accessed 2 February 2015), the user has to define process functions and deal with Python keywords such as '*with*' and '*yield*' that may baffle even experienced Python programmers. Using SimPy is a big step over using plain Python for simulations, because the parallel nature of the task is highly enhanced. However, the fact that SimPy is not focused on giving prefabricated simulation components, like most COTS software alternatives do, makes it difficult for non-experienced coders to use. This is the gap that ManPy aspires to fill for manufacturing environments; however, ManPy has been developed in a generic manner to allow it to be used in other areas, for example, modelling of health care systems. Figure 1 gives a graphical depiction of the semantic transition from SimPy to ManPy.

This section has given a brief introduction to SimPy for the requirements of this article. For more information on this project, the reader is referred to its website that offers several documents. Additionally, Matloff [24, 44] provides an excellent tutorial on SimPy. Unfortunately, it is written for SimPy2, so examples are not applicable to the new version and of course it does not cover new features. Nevertheless, it still provides valuable information for DES coding in Python and SimPy.

4. MANPY

As we already stated, ManPy stands for 'Manufacturing in Python' and is a library of manufacturing objects written exclusively in Python, making use of the SimPy library. In this section, we describe how ManPy is built and how it can be utilized to model manufacturing systems. We start explaining our design decisions. Then we describe the architecture of ManPy code in a class hierarchy. We continue by giving a number of examples of how ManPy objects can be used and also customized.

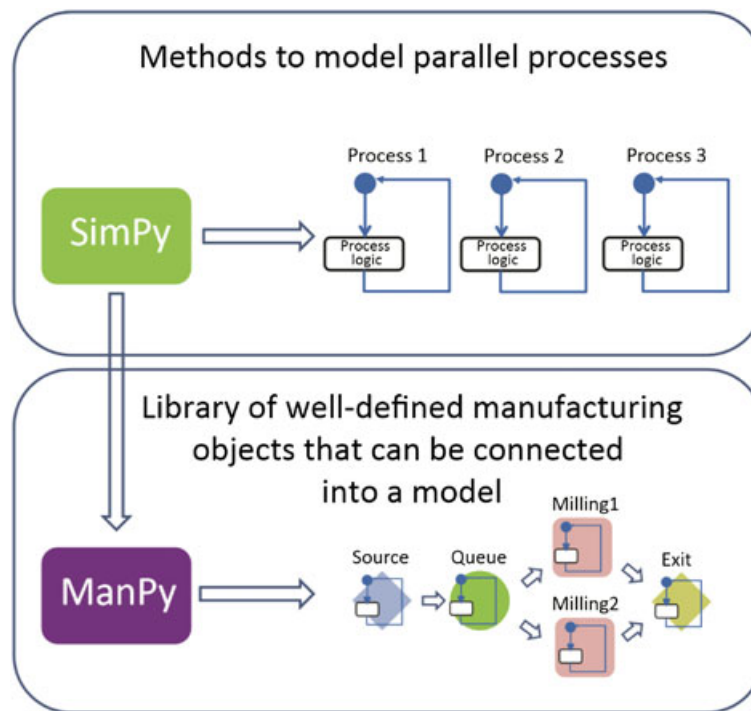


Figure 1. From SimPy to ManPy.

4.1. Design

The scope behind ManPy's design was to offer objects, that is, classes consisting of code, which a user will perceive in a similar way he views the objects he/she drags and drops in COTS simulation software tools that implement the process interaction world DES view [45]. Simulation objects that a user usually sees in the animated GUI of COTS software can be classified semantically into three major categories:

- Permanent objects that have a fixed position in the model. These are mostly servers of any kind, buffers and entry or exit points.
- Permanent objects that move in the model. These are resources that other objects compete for. So, for example, an operator can be a permanent resource in a production line, but at a given simulation time, he may work on different stations.
- Temporary objects that move in the model. These enter the model at some point of the simulation time, get buffered and processed in different stations and then exit the system. They may model parts getting processed in a production line, customers in a super market, patients in a clinic and so on.

Most DES software regard all of the previous categories as processes in its internals [46]. This is also the typical use of SimPy [24] that would consider both a customer and a cashier as a process. Nevertheless, an IE who uses proprietary simulation software gets firstly acquainted with the permanent objects, which he or she drags and drops in a model window and connects them with arrows to define the flow of temporary objects in the model. The first advanced customizations that a user typically tries are in permanent objects, that is, servers. For example if a modeller wants to extend the default behaviour of the customer when he or she reaches a server, he or she would do it through the server. In Plant Simulation [47], this would be most probably by adding a method in the *Entrance control* of *SingleProc* that is the Plant basic element to model servers. In Simul8 (<http://www.simul8.com/> accessed 2 February 2015), one would also change the *Routing in* actions of an *Activity*, which is the basic Simul8 component to model servers. It is indicative that in both software packages, the user, in order to override default behaviour, has to write some code.

ManPy is designed to offer a similar world view in an OS coding environment. The user gets prefabricated components that he or she is able to connect as black boxes in order to form a model. Of course, taking ManPy as a standalone project, the user would still have to create the model programmatically. But, as we will see, no coding of functions or methods should be needed, just declaration of the instances and their connections and also simulation parameters (e.g. length of an experiment). Furthermore, because the objects are modular, the simulation engine is easily integrated with any third-party user interface; this allows the user to define the model graphically and create the ManPy configuration in the background.

The objects implement a well-defined API of methods so that an advanced user can easily customize their behaviour in specific occasions, for example, at the event of a moving object entering into a server as stated in the previous text. A more advanced user can follow the conventions of the API to code completely new objects and incorporate them into the platform.

So, the design is inspired by COTS simulation software, yet it implies some basic differences. The platform is targeted at the provision of prefabricated simulation components, but this is performed in a completely OS framework rendering the capability of populating the repository and exchanging objects. Of course, the difficulty here is to provide means of describing an object's specific logic so that it can really become usable to a third party. Another difference to COTS is that ManPy is a standalone simulation library. Even though it can be connected to a GUI or some other module, it has no dependency with those. In this aspect, it is more flexible in interfacing with software or being deployed on the Web.

4.2. Architecture

Discrete event simulation is traditionally an appropriate paradigm where object-oriented (OO) programming can be used [48]. It is indicative that the ALGOL-based simulation language Simula was historically the first that introduced the class concept [49]. In order to achieve the goals set in the design, ManPy needs to implement a structured OO architecture.

ManPy architecture is depicted in Figure 2. As we see, the high-level architecture consists of four layers. The basis consists of pre-existing technology that includes Python, SimPy and other Python libraries such as NumPy [15] that is used for numerical computations or xlrd and xlwt [35] that allow reading from and writing into spreadsheet (*xls*) formats. It is important to state that the ManPy setup handles all these dependencies. So, a user of a Python-enabled computer can instal ManPy and its dependencies at once.

Then, there is a layer of abstract ManPy classes. A generic *ManPyObject* is the one that all ManPy simulation objects inherit from. These simulation objects are grouped into four abstract classes:

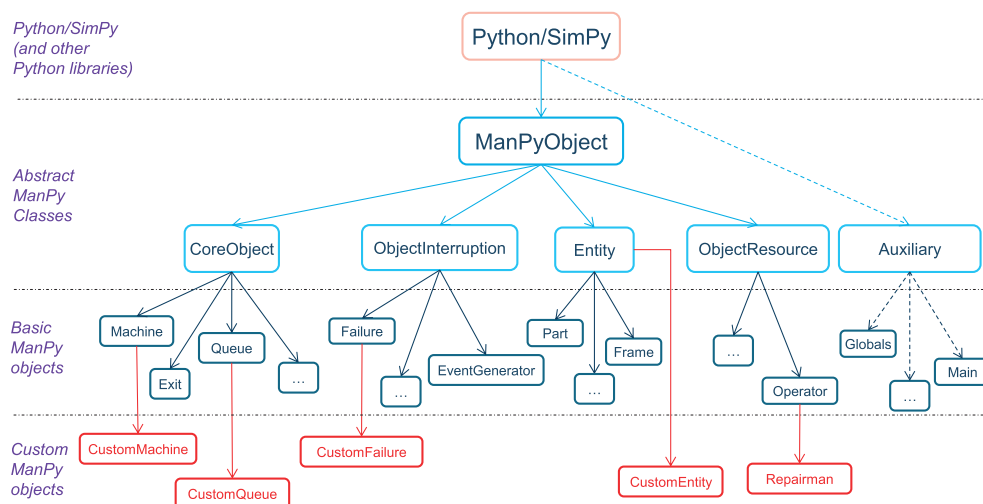


Figure 2. ManPy class hierarchy.

- *CoreObject*: all the permanent stations of the model. These can be servers or buffers of any type and also entry and exit points.
- *ObjectInterruption*: all the objects that can affect the availability of another object. For example, failures, scheduled breaks and shifts.
- *Entity*: all objects that get processed by or wait in *CoreObjects* and that are not permanent in a model. For example, parts in a production line, customers in a shop and calls in a call centre.
- *ObjectResource*: all the resources that might be necessary for certain operations. For example, repairman, operator and electric power. An *ObjectResource* is necessary in modelling when two or more *CoreObjects* compete for the same resource (e.g. two machines competing for the same operator).

Additionally, we have one more category in the abstract level that we denote as *Auxiliary*. These are not simulation objects and therefore do not inherit from the *ManPyObject*, but classes and methods needed for different simulation or coding functionalities. For example, a class enhances random number and random variable generation. Unlike the other categories described here, auxiliary components are not necessary classes nor do they inherit from one parent class. This category is more of a grouping, so we describe this relationship with dashed arrows.

By the term ‘abstract’, we mean that these classes are not meant to be instantiated. *CoreObject* is an infrastructure that provides basic methods to build objects like machines to be used in a model. However, the class itself is not designed to be used in a model as is. The same applies to *ObjectInterruption*, *Entity* and *ObjectResource*. This architecture is an attempt to abstract the simulation model into a set of *CoreObjects* that exchange *Entities*. We refer to this exchange process as ‘transaction’. Both *CoreObjects* and *Entities* may need *ObjectResources* for certain operations, and *ObjectInterruptions* may affect the availability of an object at a given simulation time. Auxiliary classes and methods help in creating and executing simulation models and interfacing with other software.

In the next level, *ManPy* objects extend the abstract classes to model a specific functionality. These classes can be used in a simulation model. Because these objects have to be able to interact like black boxes, *ManPy* offers an API of methods to model the behaviour of the object. It is out of the scope of the paper to give a complete list of this API. The reader is referred to *ManPy* documentation in GitHub. As an example, we list in the succeeding text the categorization of methods the *CoreObjects* implement:

- Definition methods*: are used for the instantiation of the object; for example, *defineRouting* defines how the object is connected with other *CoreObjects*.
- Signalling methods*: objects communicate sending signals to one another. These methods handle the communication; for example, *signalReceiver* will send a signal to the receiver in a transaction.
- Transaction methods*: are used to define how the objects exchange *Entities*; for example, *getEntity* is invoked each time a *CoreObject* receives an *Entity*.
- State control methods*: are used to retrieve the state of an object; for example, *canAccept* returns true if the object is in the state to receive an entity. This may follow a different logic for different types of objects. For example, a standard server can accept entities only if it is idle, a standard buffer only if not all of its capacity is taken or a standard exit point that is able to accept an *Entity*.
- Flow control methods*: used to control the flow of information and more specifically route the entities through the network of objects. For example, *selectReceiver* is a static method that chooses from a list of possible receivers for an *Entity* that the *CoreObject* holds and is in the state to dispose.
- Supplementary methods*: are used to define certain objects in transaction or control methods. For example, *getGiverObject* returns the giver in a transaction.
- Output and calculation methods*: are used either to output results or trace in different formats or to make certain calculations. For example, *calculateProcessingTime* returns the processing time that a server needs for an entity. In a standard Machine, this may be modelled by a distribution. In a job-shop environment though, a Machine will have to define the processing time depending on the attributes of the *Entity* being processed.

- viii. *Main simulation method*: one or more methods that are used to control the progress of the object in simulated time. In Python terms, this is a generator method. In SimPy2, there could be only one generator method where the *yield* command could be invoked, while in SimPy3, there can be more than one generator method in the same class, which provides more flexibility. The naming convention is that every CoreObject should implement a *run* method that is the main loop that is followed throughout the simulation.

The API is not limited to methods, but also a nomenclature of attributes. For example, all Entity types have a *currentStation* attribute, which holds the current position of the Entity at any simulation time. A CoreObject updates this attribute in its *getEntity* method.

ManPy objects utilize the API in order to form black boxes that have a certain behaviour as the simulation time progresses in a model. In Figure 3, we can view the event activity diagram of a standard ManPy Machine, which is implemented in its main simulation method (*run*). What is important to point out is that the IE does not require understanding of this diagram in Figure 3 to use ManPy. They only need to add this object into a model, and this will work no matter what the type of the predecessor or the successor used, for example, another Machine, a buffer, an assembly station or an exit point. Understanding of the activity diagram is only important for an SU that wants to create completely new objects. In Figure 3, methods are considered as threads that receive the control of the simulation programme in turn according to the events they are yielding for. In the figure, it can be noticed that each thread represents a CoreObject and each CoreObject has predecessor and successor objects based on the routing defined by the modelling topography. Those CoreObjects exchange with their immediate neighbours (successors and predecessors) signals (events), as well as Entities objects, which can be parts, jobs, customers and so on.

As we stated in the previous text, these different objects share a specific naming convention of methods that help them interact, for example, *getEntity*, *removeEntity*, *signalReceiver* and *signalGiver*. In Figure 3, the use of these common methods is easily noticeable. As the main thread of Machine is initiated, it releases control of the programme till there is an entity available to be delivered to it (*isRequested*). As soon as a signal is sent from its predecessor that an entity is available for delivery, it resumes control of the programme and invokes the *removeEntity* method of the predecessor through the use of its own *getEntity* method and takes control of the Entity. Consequently, it calculates the

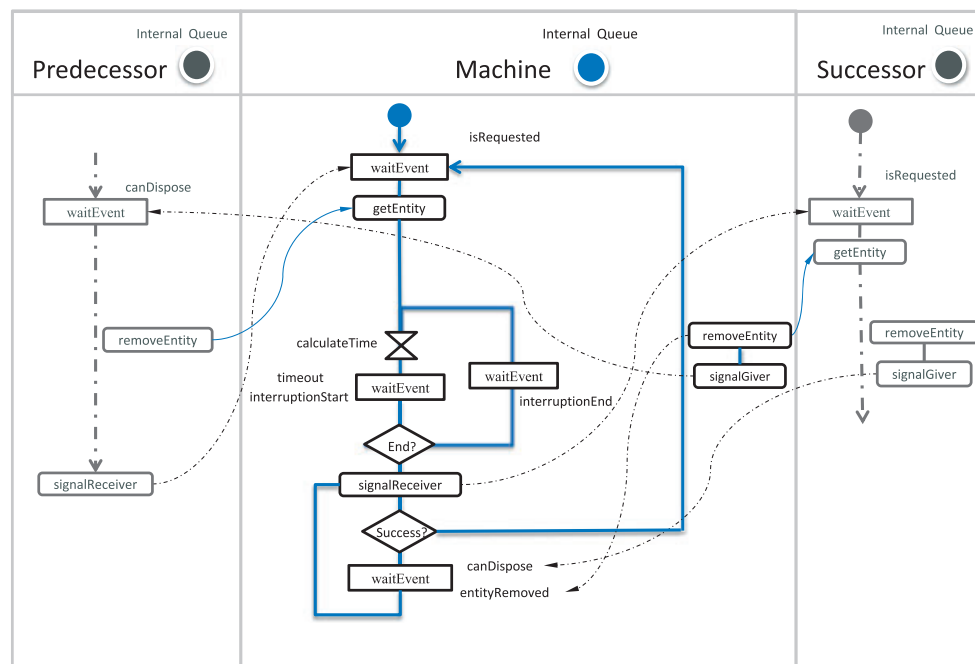


Figure 3. Event activity diagram of machine object.

processing time, according to what the modeller defined, and yields the control of the programme till processing ends (timeout event). In case of interruption (*interruptionStart*), a loop that contains a yield for *interruptionEnd* events is introduced to the flow of the programme. After processing is ended, the CoreObject tries to inform the successor making use of the *signalReceiver* method. If the signalling is not successful, the station will yield the control till it receives a *canDispose* or *entityRemoved* signal, which will allow it to regain control of the programme and try to signal the successor again if it is needed, while still holding the Entity in its internal queue. As soon as the successor receives the control, after receiving an *isRequested* signal, it will in a similar fashion receive the entity from the machine via the use of the machine's *removeEntity* method. The same method will invoke the *signalGiver* method of the machine to inform the machine's predecessor that the machine is now ready to receive a new entity.

In the final level of the architecture, there are custom objects that extend the basic ManPy objects. We should note here what is considered basic and what is customized is subjective and there may be different levels of customization. For example, MachineJobShop may extend the standard Machine in order to model a Machine that acts in a job-shop environment. Nevertheless, a simulation user who models often such environments may consider it as a basic object and possibly extend it to model more specific job-shop stations. This implies the idea of different object repositories for different domains or users that we will elaborate on further in Section 7.

In the following sections, we describe simple examples of how this architecture can be used to develop simulation models of basic or custom ManPy objects.

4.3. Basic ManPy objects

In the Auxiliary Abstract Class is the *main* script, which is used to develop a DES model. The ManPy *main* script performs the following operations:

1. reads or defines the objects;
2. creates the objects;
3. defines the structure and sets the topology of the model;
4. in every replication:
 - a. initializes the simulation, the objects and sets the work in progress;
 - b. activates the objects;
 - c. runs the simulation; and
 - d. conducts post-processing operations; and
5. after the simulation is over, outputs the results in a desirable way.

A *main* script is separate from the simulation objects used within a DES model, so users may define their own *main* script, which may constitute interfaces with other software, for example, an excel spreadsheet or a drag-and-drop GUI. Main scripts can also be small pieces of raw Python code like the examples documented in the succeeding text.

This example will present how a model of basic ManPy objects can be developed and executed. We will examine the simple problem of a single server, a production line that consists only from a point of entry (Source), one buffer (Queue), one server (Machine) and a point of exit (Exit), connected serially (Figure 4). Note that the code of the examples also exists in the OS GitHub repository where the project is being developed.

We have the following elements for the problem:

- The Source produces parts. One part is produced every 30 s.
- The Queue has capacity of 1.
- The Machine processes one part at a time. The processing time is 15 s.
- We want to study the system for 24 h.

In order to write the model, the user has first to import the ManPy objects and methods that are going to be used:

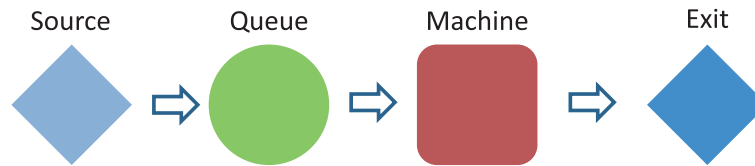


Figure 4. Single server model.

```

from dream.simulation.imports import Source, Queue, Machine, Exit
from dream.simulation.Globals import runSimulation

```

In the first line, we manually imported the four simulation objects that the model will use. The user can also use `from dream.simulation.imports import *` to import all ManPy objects, but it requires less resources to import only what is needed. In the second line, we import the *Globals.runSimulation* method. This is a supplementary method that is used to run the simulation.

Then the simulation objects should be defined:

```

# define the objects of the model
S=Source('S1', 'Source', interarrivalTime={'Fixed':{'mean':0.5}}, entity='Dream.Part')
Q=Queue('Q1', 'Queue', capacity=1)
M=Machine('M1', 'Machine', processingTime={'Fixed':{'mean':0.5}}),
E=Exit('E1', 'Exit')

```

In every *CoreObject*, the first two arguments are an identifier (*ManPyObject.id*, e.g. 'S1') and a name (*ManPyObject.objName*, e.g. 'Source'), both defined as strings, which are also the only mandatory ones. Time distributions like processing or interarrival times are defined as dictionaries, giving thus more flexibility to the attributes required. ManPy time units are decimals. So 30 and 15 s have been translated to 0.5 and 0.25 min respectively. As we see in the Source, we define also the Entity type that is created.

In the following, we need to define the connection between the objects. This is achieved by:

```

# define predecessors and successors for the objects
S.defineRouting(successorList=[Q])
Q.defineRouting(predecessorList=[S],successorList=[M])
M.defineRouting(predecessorList=[Q],successorList=[E])
E.defineRouting(predecessorList=[M])

```

Every *CoreObject* implements *defineRouting* as one of its definition methods. Note that the method of the class (e.g. Source) is called on the instance (e.g. S) and in the arguments other instances may be used (e.g. Q). Also, all *CoreObjects* have in their attributes a list that holds all the possible successors (*successorList*) and one to keep all the predecessors (*predecessorList*). Of course logical exceptions exist, e.g. no successor can be defined for an Exit type.

Now that the model is fully defined, we can run the simulation using the *runSimulation* method that we previously imported:

```

# call the runSimulation giving the objects and the length of the experiment
runSimulation(objectList=[S,Q,M,E], maxSimTime=1440.0)

```

We gave two arguments in the method: Firstly, a list with all the simulation objects. Then we provided the length of the experiment. ManPy needs an abstract time unit that is defined by the user. In this model, we picked minutes, which was used also in the object definitions. The length of the simulation is set to 1440.0 that correspond to 24 h. The *runSimulation* method can take more arguments, such as *numberOfReplications*, that defines how many times we want the simulation to be run. The default value is 1, but in stochastic cases, we need many replications to be able to statistically evaluate the results. The method conducts the following operations for every replication:

- Create a SimPy environment where the objects are modelled.
- Initialize the simulation objects.
- Activate the simulation objects.
- Run the simulation.

- Conduct post-processing actions. For example, generally the working time of a Machine is increased as soon as a processing operation ends. If the Machine is in the middle of processing a part by the end of the simulation time, then this processing time will be calculated in the post-processing and added to the total working time.

Finally, we may select how to output results. For example:

```
print "the system produced", E.numOfExits, "parts"
working_ratio=(M.totalWorkingTime/1440.0)*100
print "the total working ratio of the Machine is", working_ratio, "%"
```

Attributes like *numOfExits* or *totalWorkingTime* are results gathered by ManPy, which are contained in the API. The output in our console will be as follows:

```
the system produced 2880 parts
the working ratio of the Machine is 50.0%
```

Given the simplicity and deterministic nature of the model presented, it is easy to establish that the result is reasonable. In the next section, we explain methods we use to verify our code either in deterministic or stochastic models. As we said, the full example is available to view and test in the project repository.

In this situation, the user did not need to write any classes or methods on his or her own, just use simple commands to declare, run the model and output results. This interaction is shown in the class diagram of Figure 5. In this figure, only the attributes and methods that the user utilized are visible. Of course in the background, many methods and attributes of the ManPy API have been utilized. For example, *runSimulation* invokes *initialize* and *postProcessing* in every object. Nevertheless, the user that obtains prefabricated objects does not need to be aware of this. We expect this kind of interaction using only prefabricated objects to be simple enough for an IE with limited expertise to build models. In the next section, we give a slightly more complex example, where the IE can customize the objects to achieve specific behaviour.

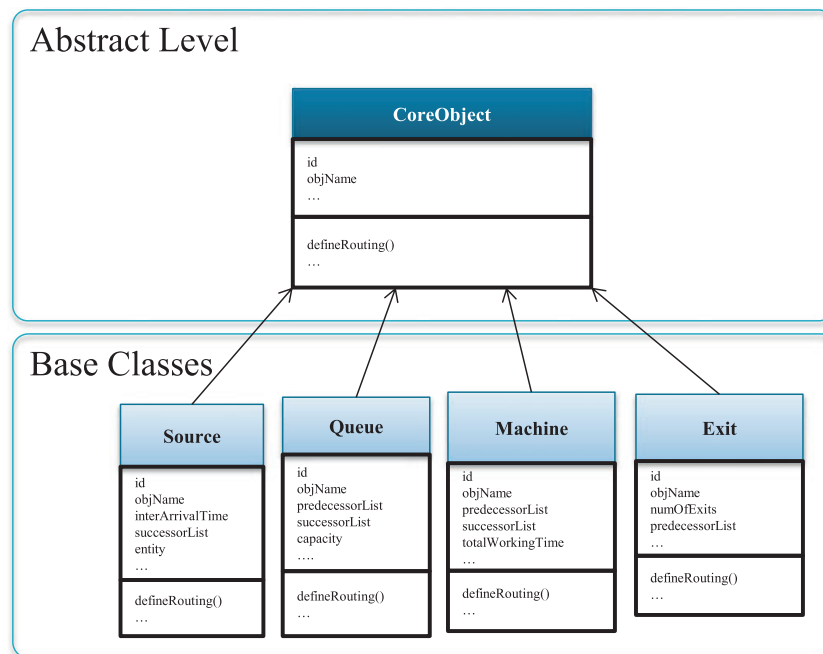


Figure 5. Class diagram of single server example.

4.4. Custom ManPy objects

We will now present a slightly more complex example that includes also customization of objects. The system has the structure shown in Figure 6. In this case, we have parts entering the system from the Source and getting queued for one of two milling Machines.

We have the following data:

- The Source produces parts. One part is produced every 30 s.
- For Milling1: processing time is fixed to 15 s, mean time to failure is 1 h and mean time to repair 5 min.
- For Milling2: processing time is fixed to 15 s, and we have no failures.
- The capacity of the Queue is infinite.
- We want to study the system for 24 h and identify the number of items that are produced and the working ratio of both Machines.

Queue is re-defined as the following:

```
Q=Queue('Q', 'Queue', capacity=float("inf"))
```

and using the objects defined in the previous example, the routing is defined as

```
Q.defineRouting([S],[M1,M2])
```

As we see, Q just receives two successors in its *successorList*. The default behaviour of a ManPy CoreObject is that if more than one successor is available to receive an Entity, it will select the one that has been idle longest.

The failures in Machine1 will be defined as a separate ObjectInterruption type object:

```
F=victim=M1, distribution={'TTF':{'Fixed':{'mean':60.0}}, 'TTR':{'Fixed':{'mean':5.0}}}
```

Running the problem, we obtain the following:

```
the system produced 2880 parts
the working ratio of Milling1 is 23.0902777778%
the working ratio of Milling2 is 26.9097222222%
```

As expected, Milling2 is slightly busier than Milling1 because it does not have failures.

Now, let us assume that in our system, the first milling station has a higher priority than the second. So, the policy is that parts will be routed to M1 unless it is busy or in failure. In order to model this, we need to create a new buffer type, which extends the method where the Queue object selects the receiver object:

```
# the custom queue
class SelectiveQueue(Queue):
    # override so that it first chooses M1 and then M2
    def selectReceiver(self, possibleReceivers=[]):
```

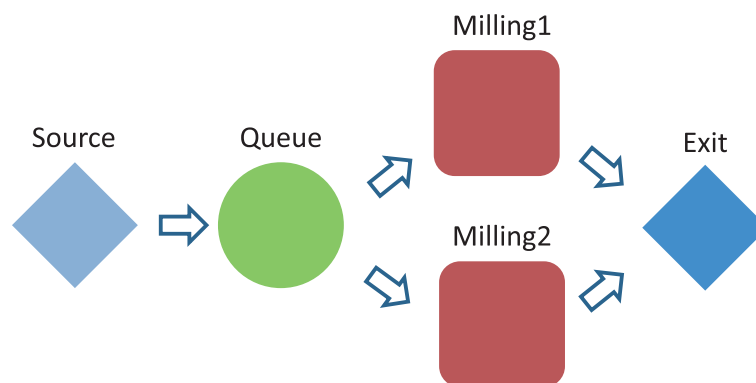


Figure 6. Parallel servers model.

```

if M1.canAccept():
    return M1
if M2.canAccept():
    return M2

```

the instance now is defined as *SelectiveQueue*
Q=*SelectiveQueue*('Q', 'Queue', capacity=float("inf"))

We see that the new object overrides the *selectReceiver* method, so that it will first check for M1. Only if M1 is unavailable it will check for M2. In order to check the availability of each object, *canAccept* (state control method) is invoked on it. *Q* is defined now as an instance of the new type *SelectiveQueue*.

Running the simulation and outputting the results we obtain the following:

```

the system produced 2880 parts
the working ratio of Milling1 is 46.1805555556%
the working ratio of Milling2 is 3.819444444444%

```

As expected, the Milling1 machine now has the highest workload.

The previous implementation is not considered generic, because the object uses specific instances (M1 and M2) in its calculations. It is easy enough to follow a more generic approach, so that the new object would sort the predecessors according to a *priority* attribute. In order to achieve that, we have to equip our Machine instances with a priority value:

```

M1.priority=10
M2.priority=0

```

The specific value is not of importance, as long as we give M1 a higher priority than the one of M2. This also demonstrates the power of Python that offers the possibility creating and parameterizing such attributes in one line.

Now, we can make a cleaner implementation of *SelectiveQueue* that uses the priority attribute in its *selectReceiver*.

```

# the custom queue
class SelectiveQueue(Queue):
    # override so that it chooses receiver according to priority
    def selectReceiver(self, possibleReceivers=[]):
        # sort the receivers according to their priority
        possibleReceivers.sort(key=lambda x: x.priority, reverse=True)
        if possibleReceivers[0].canAccept():
            return possibleReceivers[0]
        elif possibleReceivers[1].canAccept():
            return possibleReceivers[1]

```

The additional complexity is that we now used Python's *sort* function in order to sort the list of possible receivers based on their priority attribute. Nevertheless, even a beginner Python programmer can find how to do this through a simple search on the Internet. This is one advantage of using a popular computer language instead of a custom one like many COTS simulation tools do (for example, Plant uses SimTalk). No matter how good the documentation and support these products offer, it is always much easier to obtain information on a language that is supported by a community of millions of users.

As expected, the model gives the same result as the one of the previous example, but this is a cleaner implementation because it does not involve specific instances. The first implementation was fully functional at the application level of a specific model, but the latter could be used for any model that we need a Queue to prioritize its successors. This implies again the idea of having different levels of customization.

As a final example, we will edit the objects so that we count exactly how many were processed by each station. First, we will create two global counters and initialize them to zero:

```

From Globals import G
G.NumM1=0
G.NumM2=0

```

G is a class to store global variables for the model. In the following, we will create a custom machine so that whenever it takes a part, it creates a new attribute in it and it gives its id as value:

```

# the custom machine
class Milling(Machine):
    def getEntity(self):
        # call the parent method to get the entity
        activeEntity=Machine.getEntity(self)
        # create an attribute to the obtained part
        # and give it the value of the object's id
        activeEntity.machineId=self.id
        # return the entity obtained
        return activeEntity

```

The Exit will be extended so that each time it receives an Entity, it reads *machineId* and updates the counters.

```

# the custom exit
class CountingExit(Exit):
    def getEntity(self):
        # call the parent method to get the entity
        activeEntity=Exit.getEntity(self)
        # check the attribute and update the counters accordingly
        if activeEntity.machineId=='M1':
            G.NumM1+=1
        elif activeEntity.machineId=='M2':
            G.NumM2+=1
        # return the entity obtained
        return activeEntity

```

Of course, now we will define M1 and M2 as Milling and E as CountingExit. In the end of the example, we also add print statements for the global attributes:

```

print M1.objName, "produced", G.NumM1, "parts"
print M2.objName, "produced", G.NumM2, "parts"

```

Running the example, the following output is printed:

```

the system produced 2880 parts
the working ratio of Milling1 is 46.1805555556%
the working ratio of Milling2 is 3.81944444444%
Milling1 produced 2660 parts
Milling2 produced 220 parts

```

This example showcased how ManPy objects can be customized to achieve a specific behaviour. The interaction is depicted graphically in Figure 7. Because of limited space, the figure demonstrates only the CoreObject echelon where custom objects were made. As we said, the example contains also an ObjectInterruption (Failure) and two global variables of the auxiliary class G. Also, attributes and methods already highlighted in the previous example are omitted. The new attribute that had to be created and the methods that had to be overridden are noted with underlined letters.

The customizations on the examples in the previous text have been minimal that an IE should be able to apply after an initial learning with the platform. Of course, as it is stated also in the introduction, the IE is expected to apply customizations within limits. It should be the advanced SU that applies complex customizations in order to provide an IE with the appropriate set of basic objects. Figure 8 demonstrates how customized ManPy CoreObjects were developed in order to provide a solution in a real industrial pilot case of a production line that is characterized of batching and sub-batching operations (Section 6). It is not in the scope of the current paper to fully explain Figure 8, because it would imply detailed description of the actual system, and the objective here is to describe ManPy in a generic way.

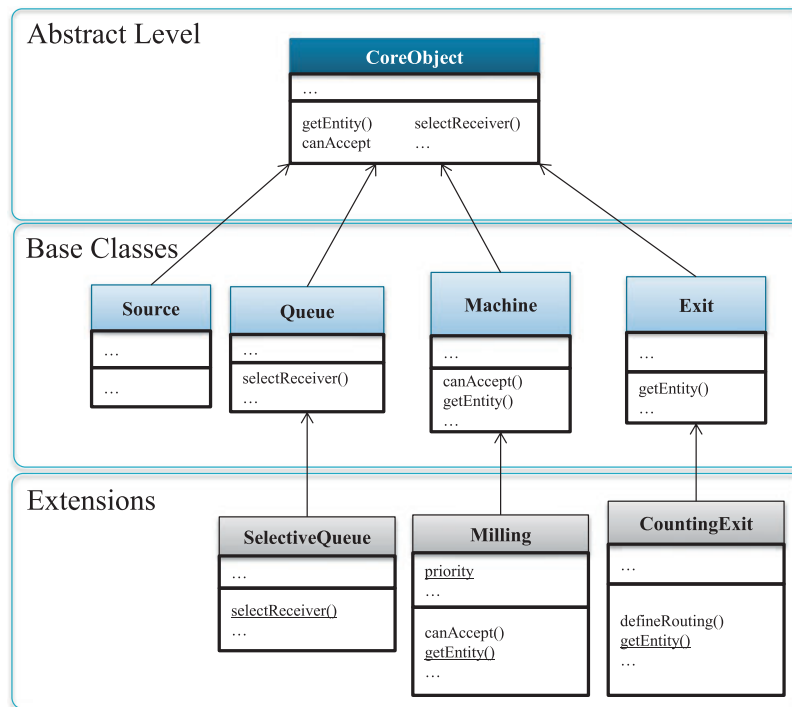


Figure 7. Class diagram of parallel server example.

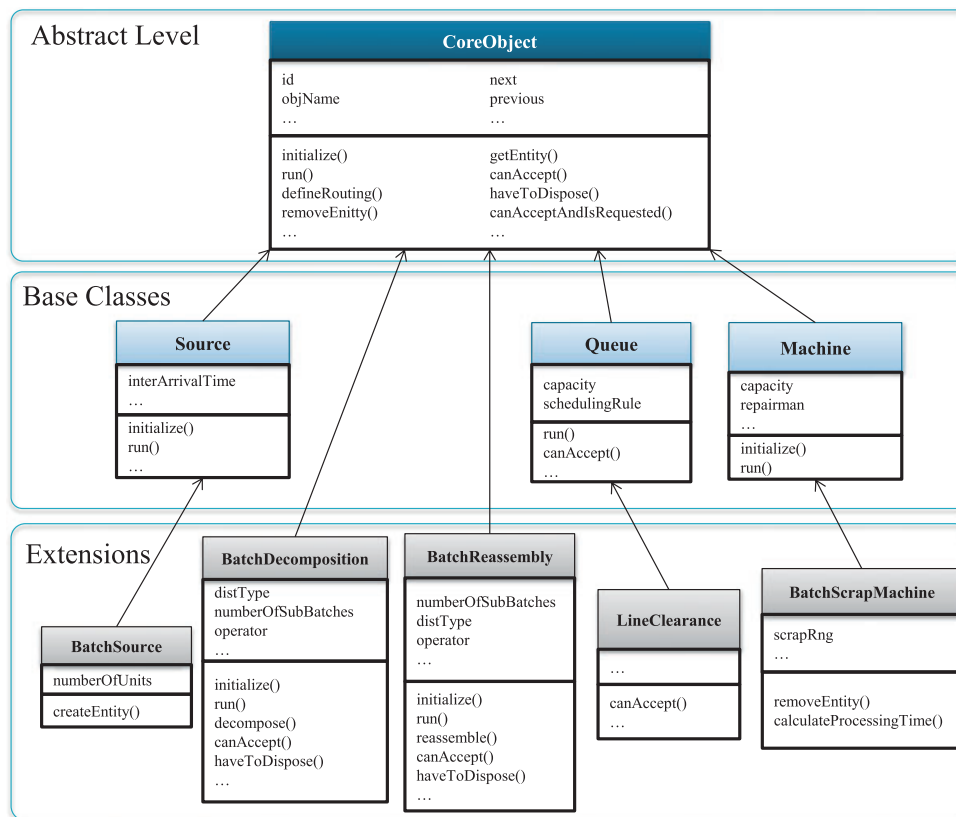


Figure 8. Class diagram of ManPy customization for a real pilot case.

5. CODE VERIFICATION

According to Shannon [1], ‘Verification seeks to show that the computer programme performs as expected and intended’. Simulation code is notoriously difficult to debug. In the ManPy world view (and in other DES programmes), we have processes moving in parallel, obtaining and suspending control of the programme at certain points of the simulation time. These transitions are triggered by events that in even simple models it is impossible for the developer to fully review. So, evaluating the code accuracy becomes extremely complex.

A primary means of verification is the provision of methods to output the simulation trace. Every ManPy object can implement its version of *outputTrace*, which outputs significant events of the simulation run that can be stored in a spreadsheet. All the objects output to the same sheet, and the events are sorted according to simulation time. The trace is used for debugging and verification of very simple models. For more complex cases, more robust means are required. In the succeeding text, we present methods we use to establish the consistency of ManPy code.

5.1. Against COTS

Commercial off-the-shelf simulation software has been around for several decades. These packages exist in a highly competitive environment, continuously striving to improve their products at all levels. Also, they currently have more industrial users than most OS alternatives; thus, they get wider testing.

Every time we build a novel ManPy object or model, we compare the results against a COTS alternative. The tool we use is Plant Simulation by Siemens [47]. Plant is a powerful package that provides the user with a highly flexible OO environment. In the left part of Figure 9, we see the model of our second example as it was built in Plant Simulation. The *EndSim* method was used to output the results at the end of the simulation as shown in the right part of the figure. In such deterministic cases, it is possible for us to establish the full accuracy of the code. Even for results that include many decimals, for example, the working ratio of a machine, we need the models to give identical results up to the last decimal.

In stochastic situations, of course, we cannot achieve the same convergence. Python uses the Mersenne Twister [50] as its core random number generator (RNG). In Plant Simulation, we could not identify which RNG is employed. Even if both two simulation software worked with the exact same RNG and seeds, they may not give identical results. The reason is that the same system may be modelled with slightly different logic in two different simulation engines. For this reason, random numbers may be produced in a different order, leading to different results.

The way to verify such models is to use confidence intervals. As an example, we change the processing time of a Machine in the ManPy model we described so that it follows the normal distribution. We set the average to 0.25 and the standard deviation to 0.01. We also truncate the distribution

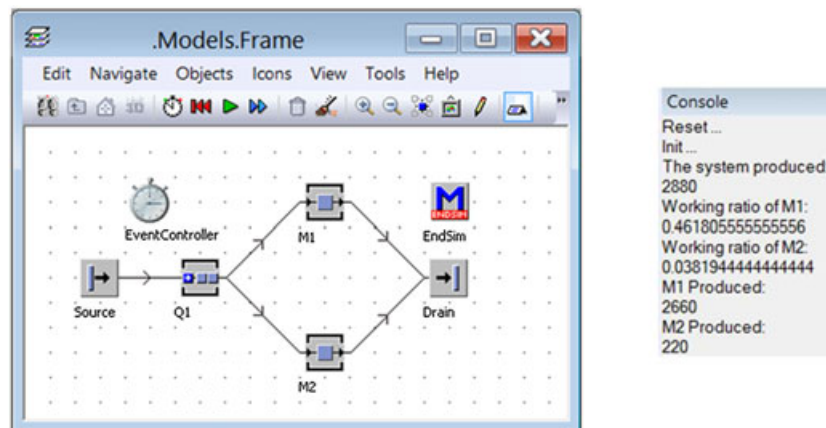


Figure 9. Plant Simulation equivalent of parallel servers' example.

setting the minimum to 0, because a processing time cannot be negative, and maximum to 1. In ManPy, such a declaration would again use the dictionary type:

```
M=Machine('M1', 'Machine',
processingTime={'distributionType': 'Normal', 'mean': 0.25, 'stdev': 0.01, 'min': 0, 'max': 1})
```

Running this model both in ManPy and in Plant for 20 replications, we take the following 95% confidence intervals (Table I).

We follow this procedure giving different attributes in the models and also testing various random number seeds. One advantage of ManPy is that, being OS, it can be connected with other means of random number generation if there is a wish to.

This is an ongoing procedure that we apply in ManPy objects and models. Table II shows a variety of such models and the purpose for which each one was built. Also, we apply a similar approach to verify models built for pilot cases [51].

5.2. Unit testing

Unit testing [52] is used in ManPy in order to establish the code's consistency in different versions. Simulation software is complex, and sometimes, a change in part of the code introduces errors that the developer cannot foresee. For this reason, each time a model is adequately verified (using the methods described in the previous text) its result is added in a test dump folder. A specific test Python script can automatically run all the tests that are already committed in the project. In essence, this script sequentially runs all the test simulation models and compares their results to the expected ones. For every test, there are three possible outcomes: *error* that means the code crashed, *fail* that means the code did not crash but the result was different than the expected and *ok* that means the model gave the expected results (see Figure 10). In the case of errors, the reason is also outputted in the console at the end of the test.

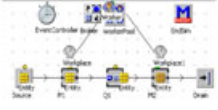
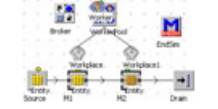
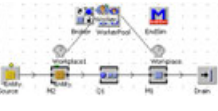
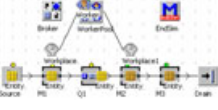
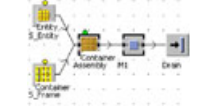
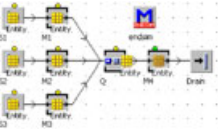
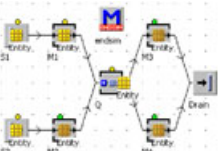
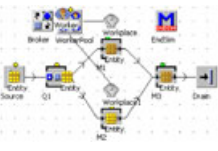
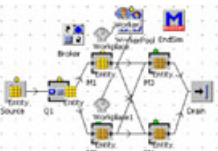
Table I. Example of confidence intervals comparison.

Tool	95% lower bound	Average	95% upper bound
ManPy	49.979187112243636	49.997680618612485	50.01617412498133
Plant Simulation	49.9709246602312	49.9954797058326	50.0200347514341

```
ok
testJobShop2RPC (dream.tests.testSimulationExamples.SimulationExamples) ... ok
testNonStarvingLine (dream.tests.testSimulationExamples.SimulationExamples) ...
ok
testNonStarvingLineBatches (dream.tests.testSimulationExamples.SimulationExample
s) ... ok
testParallelServers1 (dream.tests.testSimulationExamples.SimulationExamples) ...
ok
testParallelServers2 (dream.tests.testSimulationExamples.SimulationExamples) ...
FAIL
testParallelServers4 (dream.tests.testSimulationExamples.SimulationExamples) ...
ok
testSerialBatchProcessing (dream.tests.testSimulationExamples.SimulationExamples
) ... ok
testServerWithShift1 (dream.tests.testSimulationExamples.SimulationExamples) ...
ok
testServerWithShift3 (dream.tests.testSimulationExamples.SimulationExamples) ...
ok
testServerWithShift4 (dream.tests.testSimulationExamples.SimulationExamples) ...
ok
testSettingWip1 (dream.tests.testSimulationExamples.SimulationExamples) ... ok
testSettingWip2 (dream.tests.testSimulationExamples.SimulationExamples) ... ok
testSettingWip3 (dream.tests.testSimulationExamples.SimulationExamples) ... ok
testSingleServer (dream.tests.testSimulationExamples.SimulationExamples) ... ok
testTwoServers (dream.tests.testSimulationExamples.SimulationExamples) ...
```

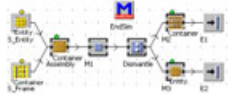
Figure 10. Example of ManPy unit testing execution.

Table II. Test models in ManPy and Plant Simulation.

Model number	Model description	Model purpose	Plant equivalent screenshot
01	Serial line. A Source is generating Parts that get processed by two Machines. Between the Machines, there is a Queue. After the second Machine, Parts depart from an Exit. The Machines can obtain failures, and they may need a Repairman in order to get fixed.	First test of connecting objects in a model	
02	The same as model 01, but the Queue is removed.	Test of connecting Machines directly one to another	
03	The same as model 01, but the positions of the two Machines are exchanged.	Test of creating a new model with the same objects	
04	The same as model 01, but after the second Machine, a third one is added.	Test of expanding a model	
05	Two Source objects generate Parts and Frames. The Frames are loaded with four Parts in an assembly object. After that, they are processed by a Machine and they exit the system.	Test of the assembly object	
06	Three Source objects create Parts that are sent in three parallel Machines. Then all three Machines send the Parts to the same Queue, and they are all processed by a fourth Machine before they exit the system.	Test of Queue object with multiple predecessors	
07	Two Source objects create Parts that are sent in two parallel Machines. Then both Machines send the Parts to the same Queue, which sends them in another two parallel Machines to get processed before they depart from the same Exit.	Test of Queue with multiple successors and Exit with multiple predecessors	
08	A Source produces Parts that firstly get into a Queue before they are sent in one of two parallel machines. Both Machines send the processed Parts to a third one.	Test of Machine with multiple predecessors	
09	The same as model 08, but also in the second level, there are two Machines in parallel.	Test of Machine with multiple predecessors and multiple successors	

(Continues)

Table II. (Continued)

Model number	Model description	Model purpose	Plant equivalent screenshot
10	The same as model 05, but before the Frames exit the system, they get to a Dismantle object where the Parts are unloaded. Parts and Frames get processed by different Machines and exit the model from different Exit objects.	Test of the Dismantle object	
11	The same as model 01, but instead of a Queue, there is a Conveyor object between the two Machines. The Conveyor carries Parts from the first to the second Machine with a certain speed.	Test of the Conveyor object	

In deterministic situations, unit testing is straightforward. In stochastic ones, we can still achieve the expected results if we statically define the random number generation seed of the model in the input. However, we found that in different platforms and operation systems, the random generation process of Python may vary, leading to failure of the test. This is a problem we try to overcome by relaxing the stochastic tests not to expect specific results but results that lie within intervals.

The unit test checks that the pre-existing code runs as expected. However, every time a new object is incorporated into the platform, there is no initial test coverage for it. So, the developer has to verify it on his/her own and then create one or more tests so that this object is also tested in the future. As a result, the number of tests increases over time. Currently, ManPy employs more than 150 tests that run in about 1 min (depending on the computer system). What is important is that the developer can promptly gain some confidence that his changes did not affect already verified code. In order to run the tests or add new ones in the dump, only one command in the shell is required.

After all the unit tests are successful, a change is committed in the version control system. In the development, we use Git (<http://git-cm.com/> accessed 4 November 2014) for version control. This provides a solid infrastructure for developers to work in parallel, keeps track of changes in different versions and provides support in being able to recognize the cause of potential problems.

6. CASE STUDIES

In our review of OS DES [14], we found that one of the main deficiencies of the domain is that projects rarely get to be utilized in real-life industrial cases. As we stated in the introduction, ManPy has been developed within a project called DREAM. The project offered pilot cases provided by industrial partners. The purpose of this paper is to present ManPy as a generic simulation engine without focusing on specifics of a model. For this reason, detailed description of the cases is scheduled for future publication work. Nevertheless, in this section, we give insight on how real-world case studies meliorated ManPy development and also why an OS tool can be advantageous in such cases.

ManPy is currently being used in three industrial cases. First is the case of a medical device industry, which involves a typical product dedicated line, where products get batched or un-batched while they flow in different stations. The case 2 model is a job-shop facility that produces moulds for a rapid-prototyping company. Finally, case 3 involves a factory that is active in the industry of textile machinery manufacturing. The three cases and their high-level attributes are summarized in Table III. Note that we use the abbreviations MNC for ‘multinational corporation’ and SME for ‘small and medium-sized enterprises’.

Table III. Overview of the pilot cases in which ManPy is being used.

Case	Type of business	Size	Type of model
1	Medical device industry	MNC	Production line
2	Rapid prototyping	SME	Job-shop
3	Manufacturing of textile machinery	SME	Fixed-increment time advance simulation

MNC, multinational corporation; SME, small and medium enterprises.

In the succeeding text, we list some indicative modelling complexities that these cases presented:

- Case 1. required Entities to flow in batches that in certain operations get sub-batched and in others the original batch is reassembled. Also, in order to abide with the ISO 13485 guidelines, a line-clearance procedure is adopted. This prevents units belonging to different production batches from accidental mixing by imposing a clear and physical segmentation of the line with respect to production batches.
- Case 2. required the development of ManPy CoreObjects and Entities that act in a job-shop environment. Also, the system involves a complex bill of materials for every order that imposes certain precedence constraints in jobs.
- Case 3. follows a fixed-increment time advance simulation logic [53] that checks the system at specific intervals and invokes an algorithm to allocate projects to resource capacity for the next time period.

All those peculiarities demanded ManPy to be extended in order to be able to model them. Some of these extensions required considerable coding effort by the SU. Nonetheless, we have to note that work of similar complexity would also be needed to model the cases if a COTS software package was used.

The main difference with ManPy is that for every case, it creates a layer of objects that might be reusable by a future community. For example, for case 2, a layer of job shop-specific components was implemented using a generic design. This layer was then further expanded to cover application-specific requirements (Figure 11). The case-specific layer may not be reusable for other applications because it relies exclusively on the modelling needs of the case study company. On the other hand, job-shop objects are developed in a generic manner so that they can cover other cases (Figure 11).

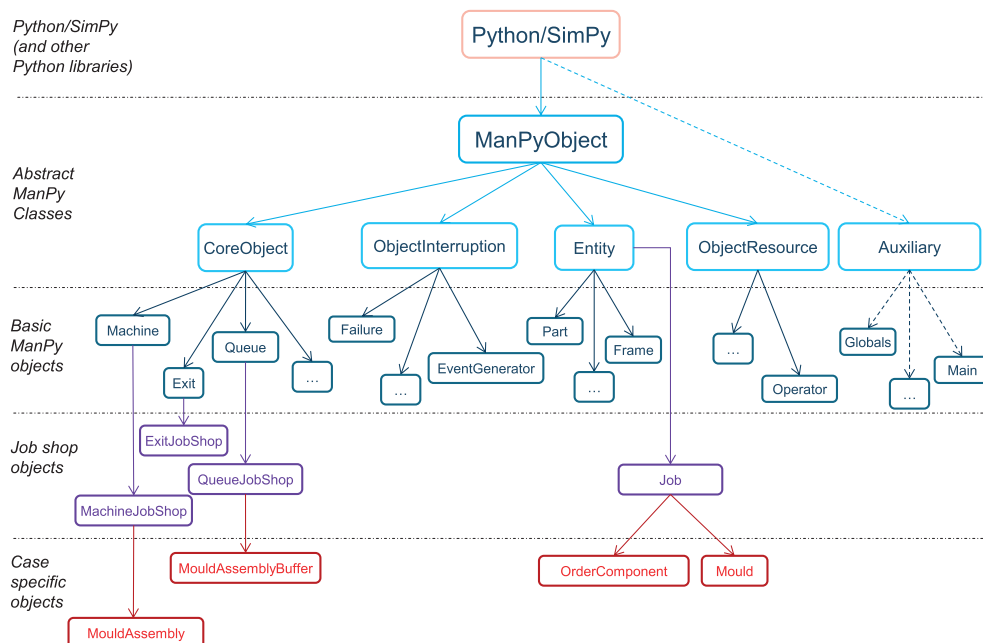


Figure 11. Extension of ManPy to generic job-shop and case-specific objects.

As more job-shop applications are implemented in ManPy, the layer will obtain more tests and become more robust. Another example is that for case 1, we designed a new buffer that inherits from a standard Queue object but applies also a constraint that is imposed by ISO specification. This means that the object can be utilized in other cases where the same ISO rules are needed.

Extensions can also be applied to COTS simulation software. Especially Plant Simulation offers an efficient OO environment where the user can make his own reusable extensions. The problem is that, no matter how generic, this work will always be limited to users that possess a license of Plant Simulation, which limits significantly the pool of SUs to create such objects. This is the main novelty that OS simulation can offer extendable prefabricated components for industrial engineers to utilize in their field.

Another asset of ManPy is that it was proven easy to interface with scenario analysis. It is reported that optimization modules are gradually becoming an integral part of DES software [54]. On the other hand, simulation software should remain modular. Being written in Python, ManPy was found flexible in interfacing with optimization routines. This was performed at two different levels, either calling an optimization routine inside the simulation run or creating an optimization routine that uses the ManPy model to stochastically evaluate the scenarios it produces. The two approaches can be employed concurrently as shown in Figure 12. Python gives the asset of easily coding optimization routines. Moreover, it contains many scientific libraries that handle such problems. For example, in case 1, ManPy interfaced with the PuLP library (<https://pythonhosted.org/PuLP/> accessed February 2 2015), which provides a linear programming framework in Python [55], in order to decide inside the simulation how the operators of the model should be allocated based on specific linear constraints.

Case studies gave also the opportunity to validate the ManPy concept. The platform was tested in large-scale models with additional complexity, which followed real-life requirements. Extensions to ManPy fell semantically inside the five echelons of architecture given in Figure 2, providing an initial proof of its completeness. Also, the fact that pilot cases were covered by objects that inherited basic ManPy components (e.g. Machine and Queue) demonstrates that the objects are really extensible and reusable.

7. OPEN SOURCE SIMULATION COMMUNITY

One very important aspect of OS development, which is by its nature distributed development, is the existence of a community of users [56]. As a project gets larger, its community gets more complex, accommodating several different levels of users [57]. The reasons behind the motivation of developers to contribute in an OS project, without having monetary compensation, constitute a subject of scientific research [58, 59]. The fact that OS is ‘free’ does not mean that the project does not

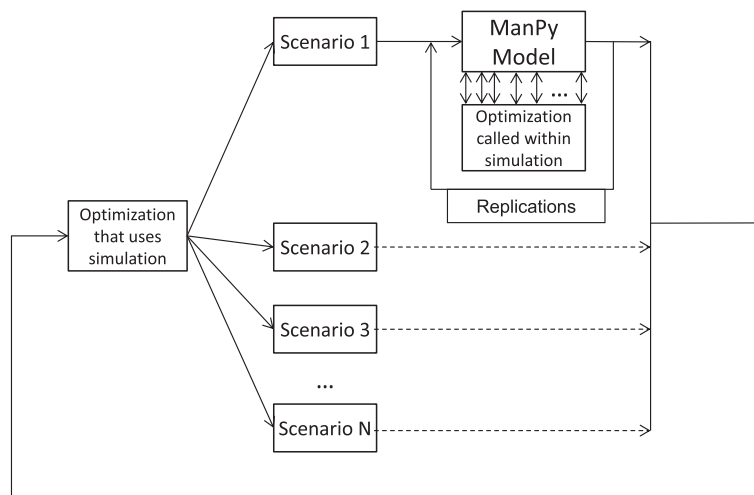


Figure 12. Interface of ManPy with optimization.

need to be managed. The necessity for control and management is equally or even more complex than it is in proprietary projects. Von Krogh *et al.* [60] state that OS software development is characterized by ‘clan control’ and governed by common values and beliefs, self-control and self-monitoring. Fitzgerald and Agerfalk [61] stress the terms mutuality and reciprocity, which pose as central aspects of the OS concept. What we found from our review of OS DES [14] is that the majority of projects fail to retain an active user community. Maybe this is partly because they neglect the managerial efforts mentioned in the previous text.

It is also of interest that the situation seems to be different in the area of computer and telecommunication networks. Indeed, tools in this domain, like OMNeT++ [22] and NS-3 [17], have succeeded in acquiring and maintaining a broad community of users. These tools have also yielded more than a thousand publications each, while the most competent generic tools we found, like JaamSim and SimPy, have only a handful of publications related to them. The fact that the domain of computer and telecommunication networks is more mature is also highlighted by the existence of review and comparison papers in this area [19, 20].

As mentioned in Section 2, ManPy aspires to draw the attention of two levels of users, an SU and an IE. Most of the projects we find seem to consider only the SU level. It is true that advanced users may also be the main code contributors who extend the platform with new elements; however, the attraction of IE users is essential for longevity of an OS project in this domain.

In the remainder of this section, we discuss issues in order to attract practitioners (IEs) and developers (SUs) into an OS DES community and we speculate how ManPy can interact with both.

7.1. Technical infrastructure

One problem that we believe deters developers from getting involved with OS DES is that many projects fail to successfully adopt and use common OS practices. One indicative example is that in several OS projects, we could not find the license under which they are distributed [14]. That would easily discourage a developer in his first interaction with a project. Also, many lack documentation or means of communication between the users. Finally, several do not employ popular OS practices like version control systems or bug trackers. Fogel [62] describes many critical factors for the success of an OS project.

ManPy is hosted in GitHub and is licensed under the terms of the Lesser General Public License. A brief *readme* file in the root of the repository informs the user about the scope of the project, its current status and how to get started. Then, ManPy documentation guides him through the first steps of installing and using the code. The Git version control system is adopted in order to facilitate distributed development. As we already described in the previous section, we have developed a testing mechanism in order to establish the consistency of the code between different versions.

The previous are primary steps that help us achieve efficient development as long as the team of contributors is small. Nonetheless, there are several other issues to consider for the future. First of all, documentation has to be actively updated and separated for different issues and users. Also, more means than traditional Portable Document Format (PDF) documents should be considered. Simulation is a traditionally complex technique [32], and the use of simulation software, no matter how well written, imposes an initial learning curve. Interactive online prototypes that allow the user to manipulate simple models and examine the results make it possible to easily comprehend the usability of the software. An online ManPy module is currently being designed, in order to help IEs to learn the tool.

Installation and deployment is often cumbersome in OS. ManPy is exclusively written in Python and uses other Python libraries like SimPy, xlwt to be able to output trace in *xls* format or NumPy [63] for scientific calculations. These may be dependent to other software. For example, NumPy depends on the ATLAS and BLAS highly optimized linear algebra packages. Both ATLAS and BLAS are written in FORTRAN, so one needs a FORTRAN compiler tool-chain to build those libraries. Furthermore, the build process is different depending on the underlying operating system used. The task of manually downloading the proper versions along with building and installing those libraries is a cumbersome and error-prone process that can be automated. An alternative would be to supply some ready-to-use software in the form of virtual machine images or software installers (executive files) with all included, but these distributions would still have to be packaged.

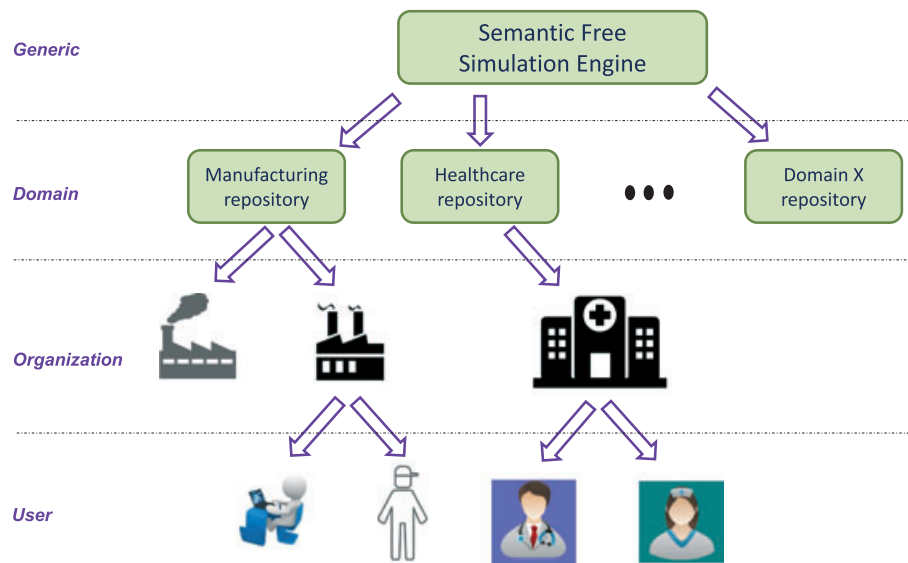


Figure 13. Levels of discrete event simulation object repositories.

Instead, we decided to build ManPy as a Python library that internally handles its dependencies. So, the user of a Python-equipped computer can, using one command in the shell, download and instal ManPy and all its dependencies and promptly obtain a working version in his or her environment. This version can be the same for someone who wants to use the software for building models or gain access in the underlying code as a contributor. This approach makes it also easier to deploy the software in a cloud of computers adopting a simulation-as-a-service approach [43] using some middleware like the Simple Language for Accounting and Provisioning Operating System [64] to automatically deploy ManPy into a number of clients. Prototypes in this direction are currently being tested and will be reported in future work [65].

7.2. Object repositories

The primary goal for ManPy is to create a layer of prefabricated objects and methods so that users can build simulation models. This is not much different than what most COTS simulation software offers. Nevertheless, what can make ManPy advantageous is that this repository of objects can be expanded at different levels. No matter how mature, the prefabricated objects that simulation software provides are never enough to easily model a real system, and additional coding is always needed. One could populate the software with more objects implementing specific logic. However, this would lead to a large number of simulation objects that would increase the learning curve and make the use of the software complex.

One way to confront this is to make a distinction in the granularity of different levels of object repositories. As its name implies, ManPy is primarily focused on manufacturing, but it is semantic-free, providing means for modelling other domains. Figure 13 summarizes our view of different levels of object repositories. First, we have the infrastructure of a semantic-free simulation engine. Then, this infrastructure is populated with domain-specific objects. It is stated that domain-specific DES environments can facilitate easier model development [66]. In the following, these repositories can be tailored into an instance specific for an organization. So a company that uses the software obtains only objects that are tailored to their type of system. Finally, we have the users inside the organization that may be IEs who construct models utilizing the tailored objects or end users that just use a tailored simulation model in their every-day decision support.

8. CONCLUSIONS AND FUTURE WORK

In this paper, we presented ManPy, an OS library of manufacturing DES objects. We started explaining why we selected to build such a tool in Python and use the SimPy library as a basis. Then a thorough description of the ManPy concept and architecture was given. Two examples have

been provided in order to give the reader an understanding of how the project can be worked on at coding level. Following, we presented mechanisms and techniques we use to verify our code and facilitate consistent development. Next, we gave a brief description of three pilot cases where ManPy is currently being deployed, stressing the benefits and the challenges of using an OS tool. Given the diversity of the systems we studied, the architecture was proven complete, meaning that the cases are handled with objects that fall semantically in the abstract concepts we introduced in Figure 2 and that basic ManPy objects can indeed get extended to model specificities. Finally, we reported on how an OS DES project in manufacturing and other domains like, for example, supply chain and health care can be successful.

Our review of OS DES [14] has shown that even though there are several competent OS tools available, there is none that has managed to gain wide recognition in the manufacturing domain. From the tools we evaluated, they require a high level of coding expertise to write simple simulation models. The goal in developing the ManPy library was to support users in embedding DES with other decision support platforms and also to support users with multiple levels of expertise (a super user and an industrial engineer) in DES.

As we explained, the effort of maintaining and disseminating such a project is far from negligible. So, issues such as code quality improvement, development of multilevel documentation and attraction of users are by default ongoing in an OS project that wishes to remain active.

Moreover, we have to validate the ManPy concept. In this paper, we discussed about code verification and presented the work conducted in order to validate the tool in industrial cases. Another echelon of validation is to check if the code is really usable by a third party. This cannot be performed via the pilot cases that we model, but only if we attract external developers to try to use ManPy. An online ManPy module is currently being designed, in order to help IEs to learn the tool.

Interfacing with other software is also an important issue. In the terms of the DREAM project, we build in parallel a GUI and a knowledge extraction tool. The former is a Web-based workflow editor that allows construction of a model graphically. The knowledge extraction tool is aimed to link production data stored in a different organization's information technology systems with the simulation software. The interface of ManPy with other modules uses formatted JSON files. In addition to enhancing usability of DREAM, this helps us also to verify the concepts of modularity and inter-connectivity of our simulation library. This work is currently being validated through the use of ManPy in the three pilot cases.

ACKNOWLEDGEMENTS

The research leading to the results presented in this paper has received funding from the European Union Seventh Framework Programme (FP7-2012-NMP-ICT-FoF) under grant agreement no. 314364.

REFERENCES

1. Shannon RE. Introduction to the art and science of simulation. In *Proceedings of the 30th Conference on Winter Simulation*, 1998; 7–14.
2. Karabuk S, Grant FH. A common medium for programming operations-research models. *IEEE Software* 2007; **24**(5):29–47. DOI: 10.1109/MS.2007.125.
3. Tako AA, Robinson S. Model development in discrete-event simulation and system dynamics: an empirical study of expert modellers. *European Journal of Operational Research* 2010; **207**(2):784–794. DOI: 10.1016/j.ejor.2010.05.011.
4. Fujimoto R. Parallel discrete event simulation. *Communications of the ACM* 1990; **33**(10):30–53. DOI: 10.1145/84537.84545.
5. Can B, Beham A, Heavey C. A comparative study of genetic algorithm components in simulation-based optimisation. In *Proceedings of the 40th Conference on Winter Simulation*, 2008; 1829–1837.
6. Seila AF. Introduction to simulation. In *Proceedings of the 27th Conference on Winter Simulation*, 1995; 7–15. DOI: 10.1145/224401.224407.
7. Banks J. Introduction to simulation. *Proceedings of the Winter Simulation Conference* 2000; **1**:9–16.

8. Tako AA, Robinson S. The application of discrete event simulation and system dynamics in the logistics and supply chain context. *Decision Support Systems* 2012; **52**(4):802–815. DOI: 10.1016/j.dss.2011.11.015.
9. Jahangirian M, Eldabi T, Naseer A, Stergioulas LK, Young T. Simulation in manufacturing and business: a review. *European Journal of Operational Research* 2010; **203**(1):1–13. DOI: 10.1016/j.ejor.2009.06.004.
10. Hlupic V. Simulation software: an Operational Research Society survey of academic and industrial users. In Proceedings of the 32nd Conference on Winter Simulation, 2000; 1676–1683.
11. Pidd M, Carvalho A. Simulation software: not the same yesterday, today or forever. *Journal of Simulation* 2006; **1**(1):7–20.
12. McLean C, Leong S. The expanding role of simulation in future manufacturing. In Proceedings of the 33rd Conference on Winter Simulation, 2001; 1478–1486.
13. King D, Harrison H. Discrete-event simulation in Java: a practitioner's experience. In Proceedings of the 2010 Conference on Grand Challenges in Modeling & Simulation, 2010; 436–441.
14. Dagkakis G, Heavey C. A review of open source discrete event simulation software for operations research. *Accepted for publication in the Journal of Simulation* 2015. DOI: 10.1057/jos.2015.9.
15. Millman KJ, Aivazis M. Python for scientists and engineers. *Computing in Science & Engineering* 2011; **13**(2):9–12.
16. Varga A, Hornig R. An overview of the OMNeT++ simulation environment. In Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems & Workshops, 2008.
17. Henderson TR, Lacage M, Riley GF, Dowell C, Kopena JB. Network simulations with the ns-3 simulator. SIGCOMM demonstration, 2008.
18. Munoz-Gea JP, Malgosa-Sanahuja J, Manzanares-Lopez P, Manzanares-Lopez P, Sanchez-Aarnoutse JC, Martinez-Rojo AM. Simulation of a P2P application using OverSim. First International Conference on Advances in Future Internet, 2009; 53–60. DOI: 10.1109/AFIN.2009.16.
19. Sarkar NI, Member S, Halim SA. A review of simulation of telecommunication networks: simulators, classification, comparison, methodologies, and recommendations. *Cyber Journals: Multidisciplinary Journals in Science and Technology-Journal of Selected Areas in Telecommunications (JSAT)* 2011; **2**(3):10–17.
20. Weingartner E, Lehn VH, Wehrle K. A performance comparison of recent network simulators. In IEEE International Conference on Communications, 2009. ICC '09:1–5. DOI: 10.1109/ICC.2009.5198657.
21. Raymond E (1999). The cathedral and the bazaar. *Knowledge, Technology & Policy* **12**(3):23–49. DOI: 10.1007/s12130-999-1026-0.
22. Varga A. The OMNeT++ discrete event simulation system. In Proceedings of the European Simulation Multiconference (ESM'2001) 9, 2001.
23. King D, Harrison H. JaamSim open-source simulation software. In Proceedings of the 2013 Grand Challenges on Modeling and Simulation Conference, 2013; 1–6.
24. Matloff N. Introduction to discrete-event simulation and the SimPy language. Davis, CA. Dept of Computer Science. University of California at Davis, 2008.
25. Núñez A, Fernández J, Filgueira R, García F, Carretero J. SIMCAN: a flexible, scalable and expandable simulation platform for modelling and simulating distributed architectures and applications. *Simulation Modelling Practice and Theory* 2012; **20**(1):12–32. DOI: 10.1016/j.simpat.2011.08.009.
26. Bause F, Buchholz P, Kriege J, Vastag S. Simulating process chain models with OMNeT++. In Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems & Workshops 19, 2008; 1–10.
27. Kriege J, Vastag S. ProC/B goes OMNeT++: efficient simulation of process chains. In Proceedings of the 14th ITG Conference on Measuring, Modelling and Evaluation of Computer and Communication Systems (MMB), 2008; 1–3.
28. Bause F, Buchholz P, Kriege J, Vastag S. A simulation environment for hierarchical process chains based on OMNeT++. *Simulation* 2010; **86**(5-6):291–309. DOI: 10.1177/0037549709104236.
29. Ueda M. Licenses of open source software and their economic values. In The 2005 Symposium on Applications and the Internet Workshops, 2005. Saint Workshops, 2005; 381–383. DOI: 10.1109/SAINTW.2005.1620054.
30. Rozel B, Viziteu M, Caire R, Hadjsaid N, Rognon JP. Towards a common model for studying critical infrastructure interdependencies. In Power and Energy Society General Meeting – Conversion and Delivery of Electrical Energy in the 21st Century, 2009; 1–6. DOI: 10.1109/PES.2008.4596465.
31. Margo MW, Yoshimoto K, Kovatch P, Andrews P. Impact of reservations on production job scheduling. In *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg: Berlin, 2008; 116–131. DOI: 10.1007/978-3-540-78699-3_7.
32. Kuljis J, Paul RJ. An appraisal of web-based simulation: whither we wander? *Simulation Practice and Theory* 2001; **9**(1-2):37–54. DOI: 10.1016/S0928-4869(01)00032-5.
33. Grandell L, Peltomäki M, Back RJ, Salakoski T. Why complicate things? Introducing programming in high school using Python. In Proceedings of the 8th Australasian Conference on Computing Education 52, 2006; 71–80.
34. Fangohr H. A comparison of C, MATLAB, and Python as teaching languages in engineering. In *Computational Science-ICCS 2004*. Springer Berlin Heidelberg: Berlin, 2004; 1210–1217. DOI: 10.1007/978-3-540-25944-2_157.
35. Oliver M. Introduction to the SciPy Stack – scientific computing tools for Python, 2013.
36. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É. Scikit-learn: machine learning in Python. *The Journal of Machine Learning Research* 2011; **12**:2825–2830.

37. McInnes A, Thorne B. ScipySim: towards distributed heterogeneous system simulation for the SciPy platform (work-in-progress). In Proceedings of the 2011 Symposium on Theory of Modeling & Simulation: DEVS Integrative M&S Symposium, 2011; 89–94.
38. Pfeiffer A, Hellerer M, Hartweg S, Otter M, Reiner M. PySimulator: a simulation and analysis environment in Python with plugin infrastructure. In Proceedings of the 9th International Modelica Conference, September 3–5, 2012.
39. Dawson B. Game scripting in Python. In Proceedings of the Game Developers Conference (GDC), 2002.
40. Tabbà F. Adding concurrency in python using a commercial processor's hardware transactional memory support. *ACM SIGARCH Computer Architecture News* 2010; **38**(5):12. DOI: 10.1145/1978907.1978911.
41. Lavrijsen WT. Optimizing Python-based ROOT I/O with PyPy's tracing just-in-time compiler. *Journal of Physics: Conference Series* 2012; **396**(5). DOI: 10.1088/1742-6596/396/5/052046.
42. Bahouth A, Crites S, Matloff N, Williamson T. Revisiting the issue of performance enhancement of discrete event simulation software. *Proceedings of the 40th Annual Simulation Symposium* 2007; **26**(28):114–122.
43. Liu X, He Q, Qiu X, Chen B, Huang K. Cloud-based computer simulation: towards planting existing simulation software into the cloud. *Simulation Modelling Practice and Theory* 2012; **26**:135–150. DOI: 10.1016/j.simpat.2012.05.001.
44. Matloff N. Advanced features of the SimPy language. Davis, CA. Dept of Computer Science. University of California at Davis, 2008.
45. Pegden CD. Advanced tutorial: overview of simulation world views. In Proceedings of the Winter Simulation Conference, 2010; 210–215.
46. Schriber T, Brunner D, Smith J. How discrete-event simulation software works and why it matters. In Proceedings of the Winter Simulation Conference, 2012; 1–15.
47. Bangsow S. *Manufacturing Simulation With Plant Simulation and Simtalk: Usage and Programming With Examples and Solutions*. Springer Berlin Heidelberg: Berlin, 2010.
48. Bischak D, Roberts S. Object-oriented simulation. In Proceedings of the 23rd Conference on Winter Simulation, 1991; 194–203.
49. Rentsch T. Object oriented programming. *ACM SIGPLAN Notices* 1982; **17**(9):51–57. DOI: 10.1145/947955.947961.
50. Matsumoto M, Nishimura T. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *Journal of ACM Transactions on Modeling and Computer Simulation (TOMACS)* 1998; **8**(1):3–30. DOI: 10.1145/272991.272995.
51. Dagkakis G, Rotondo A, Papagiannopoulos I, Heavy C, Geraghty J, Young P. From COTS simulation software to an open-source platform: a use case in the medical device industry. In Proceedings of the 8th International Conference on Digital Enterprise Technology (DET 2014), 2014.
52. Zhu H, Hall PAV, May JHR. Software unit test coverage and adequacy. *Journal of ACM Computing Surveys (CSUR)* 1997; **29**(4):366–427. DOI: 10.1145/267580.267590.
53. Kelton W, Law A. *Simulation Modeling and Analysis*. McGraw-Hill: Boston, 2000.
54. Fu MC. Optimization for simulation: theory vs. practice. *INFORMS Journal on Computing* 2002; **14**(3):192–215. DOI: 10.1287/ijoc.14.3.192.113.
55. Mitchell S. An introduction to pulp for Python programmers. *Python Papers Monograph* 2009; **1**:1–12.
56. Ye Y, Kishida K. Toward an understanding of the motivation of open source software developers. In Proceedings of the 25th International Conference on Software Engineering, 2003; 419–429. DOI: 10.1109/ICSE.2003.1201220.
57. Di Bella E, Sillitti A, Succi G. A multivariate classification of open source developers. *Information Sciences* 2013; **221**:72–83. DOI: 10.1016/j.ins.2012.09.031.
58. Lakhani KR, Von Hippel E. How open source software works: “free” user-to-user assistance. *Research Policy* 2003; **32**(6):923–943. DOI: 10.1016/S0048-7333(02)00095-1.
59. Hertel G, Niedner S, Herrmann S. Motivation of software developers in open source projects: an Internet-based survey of contributors to the Linux kernel. *Research Policy* 2003; **32**(7):1159–1177. DOI: 10.1016/S0048-7333(03)00047-7.
60. Von Krogh G, Haefliger S, Spaeth S, Wallin MW. Carrots and rainbows: motivation and social practice in open source software development. *MIS Quarterly* 2012; **36**(2):649–676.
61. Fitzgerald B, Agerfalk PJ. Outsourcing to an unknown workforce: exploring opensourcing as a global sourcing strategy. *MIS Quarterly* 2010; **32**(2):385–409.
62. Fogel K. *Producing Open Source Software: How to Run a Successful Free Software Project*. O'Reilly Media, Inc: Sebastopol, CA, United States of America, 2005.
63. Oliphant TE. *A Guide to NumPy*. Trelgol Publishing USA, 2006.
64. Smets-Solanes JP, Cerin C, Courteaud R. Slapos: a multi-purpose distributed cloud operating system based on an ERP billing model. In Proceedings of the IEEE International Conference on Services Computing (SCC), 2011; 765–766. DOI: 10.1109/SCC.2011.97.
65. Heavey C, Dagkakis G, Barlas P, Papagiannopoulos I, Robin S, Mariani M, Perrin J. Development of an open-source discrete event simulation cloud enabled platform. In Proceedings of the Winter Simulation Conference, 2014; 2824–2835.
66. Valentin EC, Verbræck A. Requirements for domain specific discrete event simulation environments. In Proceedings of the 2005 Winter Simulation Conference, 2005; 654–663. DOI: 10.1109/WSC.2005.1574306.